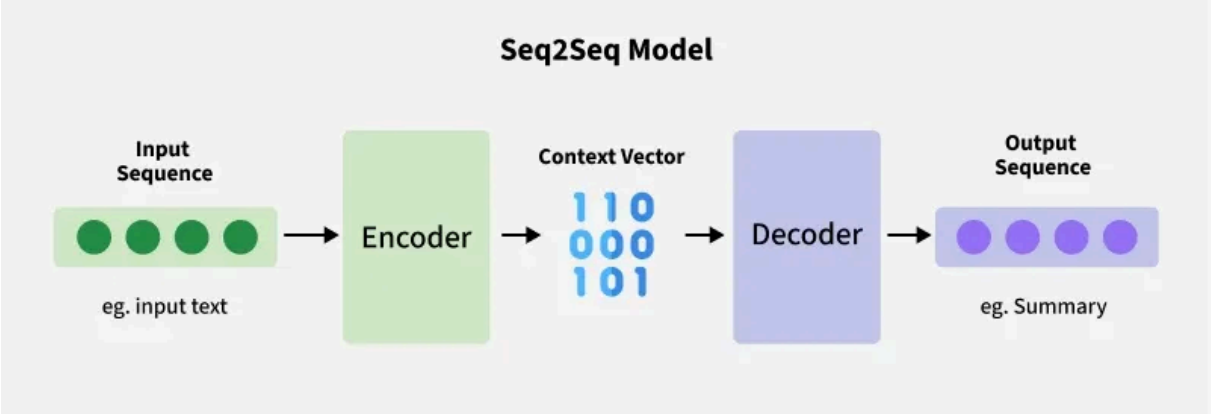
seq2seq Model

Last Updated : 10 Jul, 2026

Sequence-to-Sequence (Seq2Seq) models are neural networks designed to transform one sequence into another, even when the input and output lengths differ and are built using encoder-decoder architecture.

- It processes an input sequence and generates a corresponding output sequence.
- Handles variable-length input and output sequences
- It is used in NLP, machine translation, speech recognition and time-series prediction.



Encoder and Decoder Stack in seq2seq model

Both the input and the output are treated as sequences of varying lengths and the model is composed of two parts:

1. Encoder:

- Processes the input sequence token by token.
- Encodes the entire sequence into a fixed-length context vector (or a series of hidden states) that summarizes the important information from the input.

2. Decoder:

- Takes the context vector as input.
- Generates the output sequence one token at a time, predicting each token based on the context vector and previously generated tokens.

The model is commonly used in tasks where there is a need to map sequences of varying lengths such as converting a sentence in one language to another or predicting a sequence of future events based on past data i.e time-series forecasting.

Seq2Seq with RNNs

In the simplest Seq2Seq model RNNs are used in both the encoder and decoder to process sequential data. For a given input sequence $(x_1, x_2, ..., x_T)$, a RNN generates a sequence of outputs $(y_1, y_2, ..., y_T)$

through iterative computation based on the following equation:

$$h_t = \sigma(W^{hx}x_t + W^{hh}h_{t-1})$$

$$y_t = W^{yh}h_t$$

Here

- h_t represents hidden state at time step t
- x_t represents input at time step t
- W_{hx} and W_{yh} represents the weight matrices
- h_{t-1} represents hidden state from the previous time step (t-1)
- σ represents the the activation function (commonly tanh for RNN hidden states).
- y_t represents output at time step t

Limitations of Vanilla RNNs:

- Vanilla RNNs struggle with long-term dependencies due to the vanishing gradient problem.
- To overcome this, advanced RNN variants like LSTM (Long Short-Term Memory) or GRU (Gated Recurrent Unit) are used in Seq2Seq models. These architectures are better at capturing long-range dependencies.

How Does the Seq2Seq Model Work?

A Sequence-to-Sequence (Seq2Seq) model consists of two primary phases: encoding the input sequence and decoding it into an output sequence.

1. Encoding the Input Sequence

- The encoder processes the input sequence token by token, updating its internal state at each step.
- After processing the entire sequence, the encoder produces a context vector i.e a fixed-length representation summarizing the important information from the input.

2. Decoding the Output Sequence

The decoder takes the context vector and generates the output sequence one token at a time. For example, in machine translation:

- **Input**: "I am learning"
- **Output**: "Je suis apprenant"

Each token is predicted based on the context vector and previously generated tokens.

3. Teacher Forcing

During training, teacher forcing is commonly used. Instead of feeding the decoder's own previous prediction as the next input, the actual target token from the training data is provided.

Benefits:

- Accelerates training
- Reduces error propagation

Teacher forcing is used only during training and not during inference, where the model relies on its own previous predictions.

Step-by-Step Seq2Seq Implementation

Step 1: Import libraries

We will import **pytorch**.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

Step 2: Encoder

We will define:

- Each input token is converted to a dense vector (embedding).
- The GRU processes the sequence one token at a time, updating its hidden state.
- The final hidden state is returned as the context vector, summarizing the input sequence.

```
class Encoder(nn.Module):
    def __init__(self, input_dim, emb_dim, hidden_dim):
        super().__init__()
        self.embedding = nn.Embedding(input_dim, emb_dim)
        self.rnn = nn.GRU(emb_dim, hidden_dim)

    def forward(self, src):
        embedded = self.embedding(src)
        outputs, hidden = self.rnn(embedded)
        return hidden
```

Step 3: Decoder

We will define the decoder:

- Takes the current input token and converts it to an embedding.
- GRU uses the previous hidden state (or context vector initially) to compute the new hidden state.
- The output is passed through a linear layer to get predicted token probabilities.

```
class Decoder(nn.Module):
    def __init__(self, output_dim, emb_dim, hidden_dim):
        super().__init__()
        self.embedding = nn.Embedding(output_dim, emb_dim)
        self.rnn = nn.GRU(emb_dim, hidden_dim)
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, input, hidden):
        input = input.unsqueeze(0)
        embedded = self.embedding(input)
```

```
        output, hidden = self.rnn(embedded, hidden)
        prediction = self.fc(output.squeeze(0))
        return prediction, hidden
```

Step 4: Seq2Seq Model with Teacher Forcing

- **Batch size & vocab size**: extracted from input and decoder.
- **Encoding**: input sequence → encoder → context vector (hidden).
- **Start token**: initialize decoder with token 0.
- **Loop over max_len:**
- Decoder predicts next token.
- top1 → token with max probability.
- Append top1 to outputs.
- **Teacher forcing**: sometimes feed true target token instead of prediction.
- **Return predictions**: concatenated sequence of token IDs.

```
class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder, device):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.device = device

    def forward(self, src, trg=None, max_len=10, teacher_forcing_ratio=0.5):
        batch_size = src.shape[1]
        trg_vocab_size = self.decoder.fc.out_features
        outputs = []

        hidden = self.encoder(src)

        input = torch.zeros(batch_size, dtype=torch.long).to(self.device)

        for t in range(max_len):
            output, hidden = self.decoder(input, hidden)
            top1 = output.argmax(1)
            outputs.append(top1.unsqueeze(0))

            if trg is not None and t < trg.shape[0] and torch.rand(1).item() < teacher_forcing_ratio:
                input = trg[t]
            else:
                input = top1

        outputs = torch.cat(outputs, dim=0)
        return outputs
```

Step 5: Usage Example with Outputs

Test with example,

- **src:** random input token IDs.
- **trg:** random target token IDs (used for teacher forcing).
- **outputs:** predicted token IDs for each sequence.
- **.T:** transpose to show batch sequences as rows.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

VOCAB_SIZE = 10
EMB_DIM = 8
```

```
HID_DIM = 16
SEQ_LEN = 5
BATCH_SIZE = 2

enc = Encoder(VOCAB_SIZE, EMB_DIM, HID_DIM)
dec = Decoder(VOCAB_SIZE, EMB_DIM, HID_DIM)
model = Seq2Seq(enc, dec, device).to(device)

src = torch.randint(1, VOCAB_SIZE, (SEQ_LEN, BATCH_SIZE)).to(device)
trg = torch.randint(1, VOCAB_SIZE, (SEQ_LEN, BATCH_SIZE)).to(device)

outputs = model(src, trg, max_len=SEQ_LEN, teacher_forcing_ratio=0.7)

print("Source sequence (input tokens):")
print(src.T)
print("\nTarget sequence (true tokens):")
print(trg.T)
print("\nPredicted sequence (model output tokens):")
print(outputs.T)
```

Output:

```
Source sequence:
tensor([[1, 7, 5, 2, 9],
        [2, 1, 5, 2, 9]])

Target sequence:
tensor([[1, 2, 5, 5, 9],
        [4, 1, 6, 6, 7]])

Predicted sequence:
tensor([[7, 1, 9, 9, 9],
        [7, 6, 1, 0, 0]])
```

Output

Applications

- Converts text between languages like English to French.
- Produces concise summaries of documents or news articles.
- Transcribes spoken language into text.
- Generates captions for images by combining visual features with sequence generation.
- Predicts future sequences based on past temporal data.

Advantages

- Handles tasks like translation, summarization and captioning with variable-length sequences.
- Ideal for sequential data like natural language, speech and time series.
- Encoder-decoder structure captures input context effectively.
- Focuses on important parts of input, improving performance for long sequences.

Disadvantages

- Requires significant resources to train and optimize.
- Hard to understand the model's decision-making process.
- Prone to overfitting without proper regularization.
- Struggles with rare words not seen during training.

Suggested Quiz 6 Questions

What is the main purpose of a Sequence-to-Sequence (Seq2Seq) model?

A To classify input data

B To predict future values in time series

C To convert a sequence of inputs into a sequence of outputs

Show More

Login to View Explanation

1/6 < Previous Next >

GeeksforGeeks 15

Explore

- Machine Learning Basics
- Python for Machine Learning
- Feature Engineering
- Supervised Learning
- Unsupervised Learning
- Model Evaluation and Tuning
- Advanced Techniques
- Machine Learning Practice
- Courses

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Advertise with us
GFG Corporate Solution
Training Program

AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Data Science
CS Subjects

GfG 160
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved